

FitBuddy – Project Design Phase

Technical Design Document & System Architecture Overview

PROJECT TITLE

FitBuddy AI Generator

CORE ENGINE

Google Gemini Models

PHASE

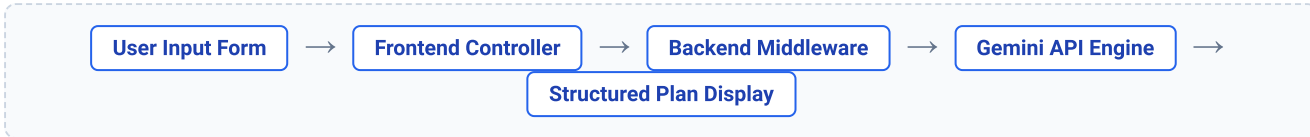
3. Project Design Phase

TARGET AUDIENCE

Developers & Evaluators

3.1 System Architecture Overview

The FitBuddy architecture follows a decoupled client-server pattern designed for ultra-low latency, clear data flow, and structured interaction with Google Gemini models.



Frontend Presentation Layer

Provides an interactive, multi-step form to collect biometric data, goals, and equipment preferences. Renders responsive daily workout schedules and diet splits.

Backend API Middleware

Validates inputs, constructs structured prompt templates, manages API key security, and formats Gemini response payloads into strict JSON schemas.

3.2 User Interface & Interaction Workflow

The interface design emphasizes simplicity and accessibility, allowing users to generate a customized fitness plan in three quick steps:

Step	User Input / Action	System Processing	UI Output
Step 1	Enter Biometrics (Age, Weight, Gender) & Primary Goal.	Calculates estimated BMR & recommended calorie targets.	Goal summary metric cards.
Step 2	Select available workout equipment & daily time budget.	Filter exercise library context for Gemini prompt setup.	Equipment tags & split selector.
Step 3	Click "Generate Fitness Plan".	Sends query to Gemini Model API with strict JSON schema.	Weekly calendar view & meal macros.

3.3 Data Model & Schema Design

The system utilizes a lightweight, structured JSON schema to exchange data between the user interface and the Gemini API model.

User Profile Payload

```
{ "age": 25, "weight": "75kg", "goal": "Muscle Gain", "equipment": ["Dumbbells", "Pull-up bar"], "days_per_week": 4 }
```

Gemini Output Schema

```
{ "workout_plan": [{ "day": "Day 1", "title": "Upper Body", "exercises": [...] }], "nutrition": { "daily_calories": 2600, "protein": "160g" } }
```

3.4 Gemini Prompt Engineering Architecture

A crucial part of the design phase is engineering system instructions and prompt templates for optimal response quality and safety.

System Instruction Structure

"You are FitBuddy, a certified personal trainer AI. Based on user parameters, generate a safe, science-backed workout routine and nutrition guideline. Always output strictly in valid JSON adhering to the specified format. Ensure rest days are properly scheduled."

3.5 Module & Directory Design

The layout for **Folder 3: Project Design Phase** within the repository contains all architectural blueprints, data models, and UI wireframes.

Design Asset File	Description	Format
architecture_diagram.png	High-level block diagram showing frontend, middleware, and Gemini API integration.	PNG Diagram
data_flow_sequence.puml	Sequence diagram illustrating request execution from user input to LLM generation.	PlantUML / SVG
prompt_schema_spec.json	JSON Schema definition used to validate output structure from Gemini models.	JSON Schema
ui_wireframes.pdf	Low-fidelity wireframes for input wizard, plan renderer, and export view.	PDF Document

3.6 Security & Performance Design Controls

API Key Protection

Gemini API credentials are restricted to server-side execution via environment variables (`process.env.GEMINI_API_KEY`), preventing exposed client keys.

Response Caching Strategy

Common baseline workout splits are cached locally to reduce API consumption and improve turnaround speed for standard fitness profiles.

3.7 Design Validation & Next Steps

The design phase directly informs the subsequent SDLC steps:

- Project Planning Phase (Folder 4):** Resource allocation, development sprint breakdown, and task assignment based on this design.
- Project Development Phase (Folder 5):** Writing code according to the architecture, prompt templates, and schema specifications outlined above.

Design Sign-off: The architecture and schema specifications defined in this document satisfy all functional requirements for the FitBuddy application.

